



Large-Scale Convex Optimization Via Saddle Point Computation

Author(s): Markku Kallio and Charles H. Rosa

Source: *Operations Research*, Jan. – Feb., 1999, Vol. 47, No. 1 (Jan. – Feb., 1999), pp. 93–101

Published by: INFORMS

Stable URL: <https://www.jstor.org/stable/222896>

JSTOR is a not-for-profit service that helps scholars, researchers, and students discover, use, and build upon a wide range of content in a trusted digital archive. We use information technology and tools to increase productivity and facilitate new forms of scholarship. For more information about JSTOR, please contact support@jstor.org.

Your use of the JSTOR archive indicates your acceptance of the Terms & Conditions of Use, available at <https://about.jstor.org/terms>



JSTOR

INFORMS is collaborating with JSTOR to digitize, preserve and extend access to *Operations Research*

LARGE-SCALE CONVEX OPTIMIZATION VIA SADDLE POINT COMPUTATION

MARKKU KALLIO AND CHARLES H. ROSA

*International Institute for Applied Systems Analysis, Laxenburg, Austria; Helsinki School of Economics, Helsinki, Finland
International Institute for Applied Systems Analysis, Laxenburg, Austria; Argonne National Laboratory, Argonne, IL 60439, USA*

(Received November 1994; revision received December 1995; accepted April 1997)

This article proposes large-scale convex optimization problems to be solved via saddle points of the standard Lagrangian. A recent approach for saddle point computation is specialized, by way of a specific perturbation technique and unique scaling method, to convex optimization problems with differentiable objective and constraint functions. In each iteration the update directions for primal and dual variables are determined by gradients of the Lagrangian. These gradients are evaluated at perturbed points that are generated from current points via auxiliary mappings. The resulting algorithm suits massively parallel computing, though in this article we consider only a serial implementation. We test a version of our code embedded within GAMS on 16 nonlinear problems, which are mainly large. These models arise from multistage optimization of economic systems. For larger problems with adequate precision requirements, our implementation appears faster than MINOS.

Large scale optimization models traditionally have been a valuable tool within the private sector and government. Their use has ranged from the planning of investments with horizons in the years, to the scheduling of more immediate day-to-day operations. The advent, in recent years, of a truly global market has increased economic competition and, as a result, made the information derived from these models of even greater value to the companies and governments using them. These factors make it increasingly important for decision makers to be able to count on algorithms that can solve their models accurately and quickly, even when the models they are working with are of very large scale. Much work has been done by researchers in all areas of applied mathematics and engineering to develop robust solution schemes for a variety of general classes of optimization problems. In this paper we will consider a new solution methodology for a very general class of convex optimization models. We will show that our algorithm possesses a variety of beneficial characteristics, such as applicability to large scale economic planning models, a structure that makes it amenable to parallelization, and no requirement that large systems of equations be solved.

We develop an algorithm for solving large scale convex optimization problems, specified as follows. Let $x = (x_j) \in R^n$ and let $f(x)$ be a convex and differentiable function. Let $g_i(x)$, for $i \in L$, be convex and differentiable functions and $g_i(x)$, for $i \in E$, be affine functions, for finite index sets L and E . Denote $g_L(x) = (g_i(x))_{i \in L}$, $g_E(x) = (g_i(x))_{i \in E}$ and $g(x) = (g_i(x)) \in R^m$ with $m = |L| + |E|$. Let vectors $l \leq u \in R^n$ define simple bounds on x . Define $X = \{x \in R^n | l \leq x \leq u\}$, which may not be bounded so that some of the l_j and u_j may have infinite values. Assume that the gradients of $f(x)$ and $g_i(x)$ are bounded on bounded sub-

sets of X . Consider the following convex programming problem:

$$\min_{x \in X} f(x) \quad (1)$$

$$g_L(x) \leq 0, \quad (2)$$

$$g_E(x) = 0. \quad (3)$$

Let $y = (y_i) \in R^m$ be the dual vector associated with the constraints (2) and (3), and let $Y = \{y \in R^m | y_i \geq 0, \forall i \in L\}$ account for the sign constraints of the dual vector y . We define the standard Lagrangian $L(x, y)$ as

$$L(x, y) = f(x) + y^T g(x), \quad (4)$$

and the Lagrangian dual problem of (1)–(3) as

$$\max_{y \in Y} \inf_{x \in X} L(x, y). \quad (5)$$

A point $(\hat{x}, \hat{y}) \in X \times Y$ is a saddle point of L over $X \times Y$ if

$$L(\hat{x}, y) \leq L(\hat{x}, \hat{y}) \leq L(x, \hat{y}) \quad \forall (x, y) \in X \times Y. \quad (6)$$

It is well known that if $(\hat{x}, \hat{y})$ is such a saddle point, then $\hat{x}$ and $\hat{y}$ are optimal solutions for (1)–(3) and (5); see, e.g., Rockafellar (1970). Furthermore, under the constraint qualification in the hypothesis of Bazaraa and Shetty's (1979) Theorem 6.2.5, such optimal solutions and saddle points are equivalent.

This class of problems has already been looked at using a variety of interior point methods (Anstreicher and Vial 1994, Nesterov 1994, Nesterov and Nemirovsky 1994). We will consider a primal-dual subgradient based approach, though, which differs significantly from an interior method in that it doesn't involve the application of a Newton-like method directly to the KKT system of equations. As such, no system of equations needs to be solved to determine

Subject classifications: Convex programming, large scale: saddle points. Parallel computation. Stochastic optimization.
Area of review: OPTIMIZATION

Operations Research
Vol. 47, No. 1, January–February 1999

each iteration's direction of search. Our method, being a form of subgradient method, will no longer exhibit the superlinear convergence that is common to interior point based methods. While we certainly do not claim that we have developed an algorithm superior to the interior point methods, our method has already proven its usefulness for large scale problems where the solution of the attendant large system of equations might be expensive. We now consider the origins of the algorithm that we will describe in this paper.

Primal-dual-based subgradient algorithms have been successfully used for solving a variety of smooth and non-smooth optimization problems (Sen and Sherali 1986; Sen and Yakowitz 1986; Rzhetskii 1994, 1990, 1991; Zabotin 1988). A unique approach of this type is Kallio and Ruszczyński's (1994a) recent linear programming algorithm, which is interpreted as a procedure for finding the saddle point for the standard Lagrangian. The key of this iterative method is to calculate the directions based on gradients of the Lagrangian at perturbed points. This procedure was extended in Kallio and Ruszczyński (1994b) to a class of saddle point problems, where the function $L(x, y)$ is convex in x and concave in y , and finite in a closed and convex set $X \times Y \subset R^n \times R^m$. Obviously, the saddle point problem (6) satisfies such requirements. Therefore, in principle, the perturbation method of Kallio and Ruszczyński (1994b) applies to convex optimization. In this article we develop suitable methods for perturbation and scaling in order to make such a method work efficiently in practice. Following Murtagh and Saunders (1987), we think of the constraints (2) and (3) as a large-scale and sparse system, which does not necessarily possess other structural properties.

In Section 1, we specialize the perturbation method of Kallio and Ruszczyński (1994b) to the saddle point problem (6). In particular, we propose a simple and computationally efficient procedure for the perturbation. In Section 2 our computer implementation together with proposed scaling procedures is presented. This implementation is embedded into GAMS (Brooke et al. 1992). Finally, preliminary numerical experience is reported in Section 3 on 16 nonlinear optimization problems arising from economic planning.

1. THE METHOD

The general idea of the saddle point algorithm (Kallio and Ruszczyński 1994b) is, in each iteration, to adjust primal and dual variables in the directions of the gradients. The adjusted points obtained are subsequently projected onto feasibility sets X and Y . These gradients are evaluated at perturbed points rather than at the solution at hand at the beginning of the iteration. For the step size, a simple rule is given to guarantee convergence. We first discuss the perturbation and propose mappings which result in an efficient implementation. Thereafter, the algorithmic steps

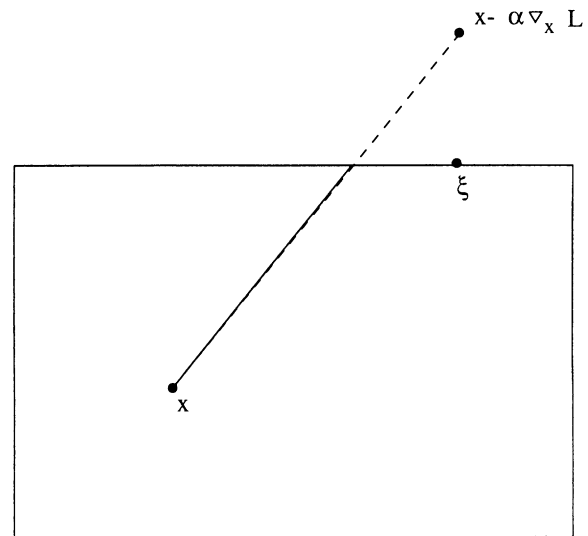


Figure 1. Projecting the gradient onto the feasible set.

are stated. Finally, convergence of the convex optimization procedure is discussed.

The general aim of perturbation is to define mappings $\xi = \xi(x, y)$ and $\eta = \eta(x, y)$ so that the gap

$$E(x, y) = L(x, \eta) - L(\xi, y), \quad (1.1)$$

is positive. Unless (x, y) is a saddle point, such a mapping always exists. We decompose $E(x, y)$ into $E_x(x, y) + E_y(x, y)$ with

$$E_x(x, y) = L(x, y) - L(\xi, y), \quad (1.2)$$

$$E_y(x, y) = L(x, \eta) - L(x, y). \quad (1.3)$$

As discussed in Kallio and Ruszczyński (1994b), there are many ways of specifying such perturbations. If the feasible sets X and Y are compact, we may define

$$\xi(x, y) \in \arg \min_{x \in X} L(x, y),$$

$$\eta(x, y) \in \arg \max_{y \in Y} L(x, y).$$

To relax the boundedness requirements, some regularizing terms may be added in the above optimization problems. In any event, due to optimization, such a perturbation can be expensive.

A more practical approach is obtained via gradient perturbation. Denote the gradients of $L(x, y)$ by $e_x = \nabla_x L(x, y)$ and $e_y = \nabla_y L(x, y)$, and define the perturbations via gradient steps:

$$\xi(x, y) = [x - \alpha e_x]_X, \quad (1.4)$$

$$\eta(x, y) = [y + \alpha e_y]_Y, \quad (1.5)$$

where $\alpha \geq 0$, and $[\]_X$ and $[\]_Y$ refer to orthogonal projections onto sets X and Y (see Figure 1).

We might then adopt line search techniques of nonlinear optimization to find a suitable value for α in (1.4)–(1.5). We begin the perturbation with some positive trial step size and apply some suitable test to determine whether the step is small enough to yield an acceptable

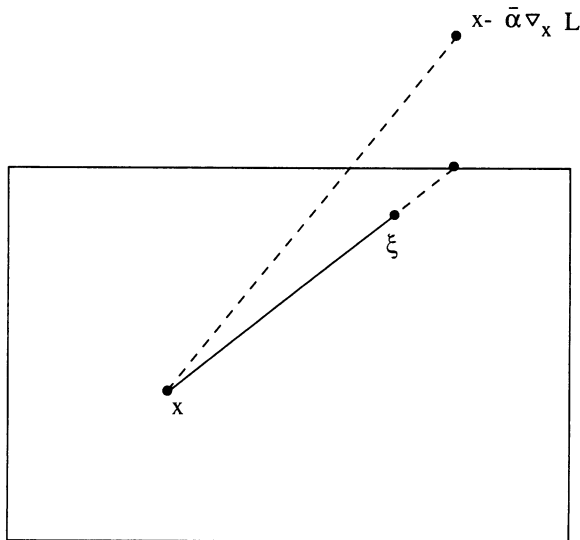


Figure 2. Determining a perturbed point ξ .

value for the gap function $E(x, y)$. If such test fails, the step size is reduced until the test is passed. Noting that the gap $E(x, y)$ as a function of α may be neither differentiable nor concave, a suitable test may be expensive to carry out. For that reason a further refinement is introduced as follows: instead of taking the perturbation direction e_x and e_y as gradients of $L(x, y)$, let $\bar{\alpha} > 0$ be a constant, e.g., $\bar{\alpha} = 1$, and define

$$e_x = -([x - \bar{\alpha}\nabla_x L(x, y)]_X - x)/\bar{\alpha}, \quad (1.6)$$

$$e_y = ([y + \bar{\alpha}\nabla_y L(x, y)]_Y - y)/\bar{\alpha}. \quad (1.7)$$

Consequently, if the perturbation is defined by (1.4) and (1.5), then the gap $E(x, y)$ becomes a concave and differentiable function for α in $(0, \bar{\alpha})$ (see Figure 2). In this interval, the Goldstein test (see, e.g., Fletcher 1987, p. 27) will be applicable to determine a suitable stepsize α for primal perturbation; see Lemma 1 below.

The Goldstein test proceeds as follows. Let $\psi(\alpha) = \psi(\alpha, x, y)$ denote the gap $E_x(x, y)$ as a function of α , and let $\psi' = \psi'(x, y)$ be the right derivative of $\psi(\alpha)$ at $\alpha = 0$. The Goldstein test is passed, if $\psi(\alpha_x) \geq \omega\alpha_x\psi'$, where $\omega \in (0, 1)$. If the test fails, a simple rule is to reduce the step size α by a factor $1 - \theta$, with $0 < \theta < 1$. A practical choice is $\omega = 0.2$ and $\theta = 0.5$.

In general, the perturbation steps α in (1.4)–(1.5) may be specified individually for each primal and dual variable. Thereby we may exploit the observation that $L(x, y)$ is linear in y , and often in practice, linear in most of the primal variables x_j as well. Besides, these step sizes may vary from one iteration to another in the saddle point algorithm. In the following, however, we restrict the discussion to the case where a single iteration dependent stepsize α_x is applied to all primal variables and another α_y is applied to the dual variables. In particular, we choose a constant step size $\alpha_y = \bar{\alpha}$ to be applied to all dual variables. For primal variables, we search for a suitable step size $\alpha_x \in [0, \bar{\alpha}]$ employing the Goldstein test. In a given

iteration of the saddle point algorithm, the step size α_x found in the preceding iteration is first upgraded by a factor $1 + \theta$ and then projected in the interval $[\bar{\alpha}, \bar{\alpha}]$, where $\bar{\alpha} > 0$ is a minimum initial value (such as 10^{-6}) applied through all iterations. The resulting value α_x is employed to begin the search for a step size in the current iteration. Following this line of thought, the perturbation mapping for (x, y) is formalized as follows.

Perturbation Mapping

Begin. Enter the perturbation routine with primal step size $\alpha_x > 0$ and dual step size $\alpha_y = \bar{\alpha}$. Replace α_x by $\min[\bar{\alpha}, \max(\bar{\alpha}, (1 + \theta)\alpha_x)]$. Find e_x and e_y according to (1.6)–(1.7).

Trial. Determine $\xi(x, y)$ by (1.4) with step size $\alpha = \alpha_x$, and $\eta(x, y)$ by (1.5) with step size $\alpha = \alpha_y$.

Completion. Let $\psi(\alpha)$ and ψ' be defined as above. If $\psi' = 0$, set $\alpha_x = 0$. Carry out the Goldstein test: if $\psi(\alpha_x) \geq \omega\alpha_x\psi'$, then the perturbation is completed. Otherwise, replace α_x by $(1 - \theta)\alpha_x$ and return to Trial.

For convergence, the following lemma will be employed.

Lemma 1. *The Perturbation Mapping satisfies the following conditions:*

1. The vectors $\xi(x, y)$ and $\eta(x, y)$ are bounded on bounded subsets of $X \times Y$.
2. $E(x, y) \geq 0 \forall (x, y) \in X \times Y$.
3. For every $(x, y) \in X \times Y$, if there is a sequence $(x^k, y^k) \rightarrow (x, y)$ such that $E(x^k, y^k) \rightarrow 0$, then (x, y) is a saddle point of L on $X \times Y$.

Proof. Condition 1 follows directly from boundedness of perturbation steps sizes and from our assumption that the gradients $\nabla f_i(x)$ and $\nabla g_i(x)$, for all i , are bounded over bounded subsets of $X \times Y$.

Observing the Goldstein test for primal perturbation, we have $E_x(x, y) = \psi(\alpha_x, x, y) \geq \omega\alpha_x\psi'(x, y)$, with $\psi'(x, y) = e_x^T \nabla_x L(x, y) \geq 0$. Thus, $E_x(x, y) \geq 0$, and $E_y(x, y) = ([y + \bar{\alpha}g(x)]_Y - y)g(x) \geq 0$, so that $E(x, y) = E_x(x, y) + E_y(x, y) \geq 0$, and Condition 2 follows.

For Condition 3, let us assume the contrary: assume that $(\bar{x}, \bar{y})$ is not a saddle point and $(x^k, y^k) \rightarrow (\bar{x}, \bar{y})$ such that $E(x^k, y^k) \rightarrow 0$. For $\epsilon > 0$, define closed neighborhoods $B_\epsilon = \{(x, y) \mid \|(x - \bar{x}, y - \bar{y})\| \leq \epsilon\}$.

If $E_y(\bar{x}, \bar{y}) > 0$, then by continuity of $E_y(x, y)$, $\inf\{E_y(x, y) \mid (x, y) \in B_\epsilon\} > 0$, for small enough $\epsilon > 0$. As $E(x, y) \geq E_y(x, y)$, we have a contradiction with $E(x^k, y^k) \rightarrow 0$.

If $E_y(\bar{x}, \bar{y}) = 0$, we show, that in a neighborhood B_ϵ , $\psi'(x, y)$ as well as the step size α_x are bounded below by a positive numbers. This together with the Goldstein condition implies that there is $\nu > 0$ such that $E_x(x, y) \geq \omega\alpha_x\psi'(x, y) > \nu$, $\forall (x, y) \in B_\epsilon$; a contradiction with $E(x^k, y^k) \rightarrow 0$.

As $(\bar{x}, \bar{y})$ is not a saddle point, $E_y(\bar{x}, \bar{y}) = 0$ implies $\psi'(\bar{x}, \bar{y}) > 0$. In (2.6), the mapping $e_x: X \times Y \rightarrow R^n$ is continuous in $X \times Y$ and $\nabla f(x)$ is continuous in X . Therefore, $\psi'(x, y) = e_x^T \nabla_x L(x, y)$ is continuous in $X \times Y$. Consequently, for small enough $\epsilon > 0$, $\inf\{\psi'(x, y) | (x, y) \in B_\epsilon\} > 0$.

Finally, to investigate possible values of α_x , define the function $\varphi(\alpha, x, y) = \psi(\alpha, x, y) - \omega\alpha\psi'(x, y)$ as the excess in the Goldstein test. Define α^* so that $\varphi(\alpha^*, \bar{x}, \bar{y}) > 0$. Then, by continuity, $\varphi(\alpha^*, x, y) > 0$, $\forall (x, y) \in B_\epsilon$, with small enough $\epsilon > 0$. As $\varphi(\alpha, x, y)$ is concave in α , for $\alpha \in [0, \bar{\alpha}]$, $\varphi(\alpha, x, y) > 0$, $\forall \alpha \in (0, \alpha^*)$, $\forall (x, y) \in B_\epsilon$, with $\epsilon > 0$ small enough. In this region, the Perturbation Mapping would not reduce the step size α_x any further. Hence, for the completion step size we have $\alpha_x \geq (1 - \theta) \min[\alpha^*, \bar{\alpha}] > 0$, for all $(x, y) \in B_\epsilon$, with $\epsilon > 0$ small enough. $\square$

The method for finding a saddle point is now stated as follows.

Saddle Point Algorithm

Initialization. Choose $x^0 \in X$, $y^0 \in Y$. Choose parameters $\bar{\alpha} \geq \bar{\alpha} > 0$, ω , θ and a stepsize parameter $\gamma \in (0, 2)$. In practise, we set $\gamma = 1$. Set $k = 0$.

Perturbation. Find perturbed points $\eta^k = \eta(x^k, y^k)$ and $\xi^k = \xi(x^k, y^k)$ employing the Perturbation Mapping.

Stopping test. Determine the gap $E_k = L(x^k, \eta^k) - L(\xi^k, y^k)$. If $E_k = 0$, then stop.

Update. Find gradients $L_x(x^k, \eta^k)$ and $L_y(\xi^k, y^k)$ and define

$$d_x^k = [-L_x(x^k, \eta^k)]_{X_k},$$

$$d_y^k = [L_y(\xi^k, y^k)]_{Y_k},$$

where X_k and Y_k are cones of feasible directions for x and y , respectively, determined by binding simple bounds at (x^k, y^k) , and $[\]_{X_k}$ and $[\]_{Y_k}$ refer to orthogonal projections on these cones.

Define

$$x^{k+1} = [x^k + \tau_k d_x^k]_{X_k}, \quad (1.8)$$

$$y^{k+1} = [y^k + \tau_k d_y^k]_{Y_k}, \quad (1.9)$$

where the stepsize τ_k is given by

$$\tau_k = \frac{\gamma E_k}{\|d_x^k\|^2 + \|d_y^k\|^2}. \quad (1.10)$$

Increase k by one and go to Perturbation.

As a consequence of Lemma 1 and the saddle point theorem of convex programming, a short proof adopted from Kallio and Ruszczyński (1994b) is given in the appendix for the following convergence result, which employs the generalized Slater constraint qualification:

Theorem 1. Assume that a saddle point of L on $X \times Y$ exists, or that an optimal solution for (1)–(3) exists, and

that $0 \in \text{int } g_E(X)$, and that $g_L(x^*) < 0$ and $g_E(x^*) = 0$, for some $x^* \in X$. Then the Saddle Point Algorithm generates a sequence $\{(x^k, y^k)\}_{k=0}^\infty$ convergent to an optimal solution of (1)–(5).

2. IMPLEMENTATION

An experimental computer code has been developed on the basis of Section 1. We call our implementation CNLP. The initial experiments indicate that scaling is crucial for an efficient implementation. We will shortly discuss the dynamic scaling procedure in CNLP. Data storage and computational steps will be outlined thereafter. Then, iterative steps are shown. Finally, we discuss the linkage of CNLP with GAMS (Brooke et al. 1992). At this point we wish to stress that all steps in CNLP have a great potential for parallelization: all primal and dual variables can be processed in parallel. Besides, sparsity can be exploited in communication as well; therefore the approach is well suited for massively parallel computing. However, our preliminary tests with CNLP will be executed in a serial computer.

Scaling

The procedure employed in the linear programming code by Kallio and Salo (1994) was adjusted for CNLP. Denote by $A = (a_{ij}) \in R^{m \times n}$ the Jacobian of $g(x)$, so that $a_{ij} = \partial g_i(x) / \partial x_j$. In each iteration, auxiliary reference quantity and value vectors $\epsilon = (\epsilon_i) \in R^m$ and $\delta = (\delta_j) \in R^n$, respectively, are defined so that

$$\epsilon_i = \sum_j |x_j a_{ij}|,$$

$$\delta_j = \sum_i |y_i a_{ij}|,$$

where x_j and y_i denote primal and dual solutions at the beginning of the iteration. If such quantities ϵ_i and δ_j are equal or close to zero, small positive safeguard values are used instead.

The scaling takes place in three phases. First, each row i of A is divided by ϵ_i and each column j by δ_j . Let $A' = (a'_{ij})$ be the resulting matrix. Let $\bar{a}_i$ be the mean of nonzero values $|a'_{ij}|$ in row i and let $\bar{a}_j$ be the corresponding column average, for each column j . Then, in the second phase of scaling, each element a'_{ij} is divided by the geometric mean of $\bar{a}_i$ and $\bar{a}_j$. In the third phase, rows and columns are scaled by Euclidean row and column norms, respectively, obtained after the second phase.

This procedure defines row and column scaling factors for the Jacobian. Primal and dual variables are scaled by the inverses of these factors. In our implementation, algebraic steps of the algorithm are applied in the scaled space, while computations are carried out in the unscaled space. It turns out that such computations employ squared scaling factors only. Therefore, there is no need for square root computations, even though the scaling procedure involves geometric means and Euclidean norms. Original problem

data are not scaled; the squared scaling factors are employed when needed.

The Data Structure

As in MINOS (Murtagh and Saunders 1987) the (potentially) nonzero data for the gradient $\nabla f(x)$ and the Jacobian A are stored columnwise, accounting for sparsity. For the purpose of dual updates, the locations of nonzero elements of A are stored row-wise as well. Bounds l and u , and the squared scaling factors are stored as dense vectors.

Iterative Steps

We now outline the computations carried out by CNLP. The steps are specified as follows.

STEP 1. To begin the first iteration, set primal variables x_j to initial values x_j^0 and project onto the simple bounds. Set dual variables y_i to zero. Choose the update step size parameter $\gamma \in (0, 2)$, the perturbation step size parameters $\bar{\alpha} \geq \bar{\alpha} > 0$ and $\omega, \theta \in (0, 1)$, and a stopping parameter $\phi > 0$ (see Step 11 below).

STEP 2. Evaluate the objective function $f(x)$, the gradient $\nabla f(x)$, the constraint function $g(x)$, and the Jacobian $A = \partial g(x)/\partial x$.

STEP 3. Find perturbation $\eta_i(x, y)$ and update row scaling factors, for all i .

STEP 4. Find a trial perturbation $\xi_j(x, y)$ and update column scaling factors, for all j .

STEP 5. Determine the gap $E(x, y)$ and carry out the Goldstein test for the primal perturbation stepsize. If the test fails, reduce the perturbation step and return to Step 4.

STEP 6. Evaluate $g_i(\xi)$, i.e., the gradient component of $L(\xi, y)$, for each i , and determine the direction d_{y_i} .

STEP 7. Evaluate the gradient component $\partial L(x, \eta)/\partial x_j$, for each j , and determine the direction d_{x_j} .

STEP 8. Determine the step size according to (1.10).

STEP 9. Update dual variables according to (1.9).

STEP 10. Update primal variables according to (1.8).

STEP 11. Evaluate an error measure

$$V(x, y) = \sum_i |y_i g_i(x)| + \sum_j |\Delta_j \nabla_j L|, \quad (2.1)$$

where $\Delta_j = \min(x_j - l_j, u_j - x_j)$ and $\nabla_j L$ is the partial derivative of the Lagrangian $L(x, y)$ with respect to x_j . If $V(x, y) > \phi|f(x)|$, return to Step 2; otherwise stop. Our experience indicates that, if $\phi = 10^{-k}$, then we may expect $k + 1$ significant digits in the optimal objective function value.

To get an idea of work per iteration, consider an application of CNLP to linear programming with ne nonzeros in A and $\nabla f(x)$. For the primal variables, both perturbation

and update steps involve an evaluation of the reduced cost vector $\nabla_x L$, each one requiring $O(ne)$ work. The same holds for updating the reference value vector. The remaining effort for the column scaling vector requires $3 \times O(ne)$ work. Summing up, the main bulk of work related to primal variables is $5 \times O(ne)$. By symmetry, the same applies to the dual, so that the total work per iteration is approximately $10 \times O(ne)$. This contrasts with the work per iteration of an interior point method which is typically $O(n^3)$, where n is the number of variables, due to the requirement that a system of equations be solved to derive a search direction.

Linkage with GAMS

GAMS (General Algebraic Modeling System) is a modeling language that enables end users to describe their convex optimization problems in a relatively clear and logical mathematical programming format, and then solve using a variety of industry standard solvers. GAMS becomes useful to optimization code developers by virtue of it being possible to attach any solver to GAMS and effectively use GAMS as a problem generator and function evaluator. This frees the developers to concern themselves solely with the solver itself. This is what we did in order to easily assess our method's applicability to existing GAMS models.

We effected the linkage via the GAMS LIB library of subroutines (Dirkse et al. 1994). This library is designed to facilitate the connection of mathematical programming solvers to GAMS. The most important feature of the library is the routine enabling the retrieval of function, gradient, and Jacobian information (NLEVAL($\cdot$)). This feature is used during each iteration of the algorithm, calculating the required gradients of the Lagrangian function that provide the directions to the perturbed points and the gradients at the perturbed points themselves. Other library routines also automatically handle such mundane but important tasks as the allocation of memory for the solver's core space, the passing back to GAMS of the state of the solution at solver termination, and the solution itself. A complete description of the library's capabilities can be obtained from Dirkse et al. (1994).

A linkage like just described greatly broadens the set of test problems available for analysis. Of course, we have also implemented a version of CNLP that runs in stand-alone mode, obtaining Jacobian and function evaluation information from separate subroutines, as in MINOS.

3. COMPUTATIONAL TESTS

CNLP was tested on 16 nonlinear problems, whose names and dimensions are given in Table I. Problems US1, ..., US16E arise from the Global 2100 model by Manne and Richels (1992). Based on this model, Rosa (1994) has developed a set of multiperiod three-stage stochastic optimization models for the U.S. energy sector. The objective is to maximize the expected present value of a logarithmic

Table I
Number of Rows, Columns, Nonzeros, and
Nonlinearities in the Test Problems

Problem	Rows	Columns	Nonzeros	Nonlinear Nonzeros
US1	398	595	1846	99
US4C	1589	2023	7376	391
US4E	2201	2449	8605	396
US8C	3177	3655	14744	775
US8E	4809	4897	18025	792
US16C	6353	6919	29480	1543
US16E	9617	9793	36049	1584
JMU	1350	1987	4958	990
LTAREF	3273	4341	13824	603
LTAPTO	3322	4341	14010	615
LTASCC	3343	4341	13954	603
LTASGE	3333	4341	13944	603
11RT	6901	7954	27320	484
ATL	5761	7102	21954	881
ATLX	6835	8520	26620	1095
ATLR	7043	8580	25379	960

utility function. The constraints include nonlinearities in production functions. Names ending with a letter E refer to formulations with explicit nonanticipativity constraints (see Rosa 1994), while those ending with the letter C refer to condensed problems with the usual block-angular structure. Thereby problem pairs US4C-US4E, US8C-US84E, and US16C-US16E are equivalent formulations. The numbers in names indicate the number of scenarios in the problem. Thus US1 is in fact a deterministic dynamic problem.

The remaining test problems are dynamic, multiregion models arising from general equilibrium models for analyses of interaction between the energy sector, the rest of the economy, and the environment. The ATL- and LTA-models are due to Manne and Richels (1995). In the LTA-models, REF is the reference case, PTO refers to a Pareto-optimal case, SGE to stabilizing carbon emissions, and SCC to stabilizing carbon concentrations. Models JMU and 11RT were developed by Olsen (1994) and Schratzenholzer (1995), respectively. For several of these test problems actual production runs are being made using CNLP.

All models were written in GAMS, and GAMS/MINOS (Brooke et al. 1992) was used for the tests. Both CNLP and MINOS employ routines provided by GAMS for function and gradient evaluations. The solution times for MINOS are obtained using default values for specs parameters with the following exceptions: (1) for some problems, the upper limit on the number of minor iterations as well as of superbasic variables was increased in order to ensure convergence; and (2) for some test runs, optimality test was replaced by the one employed in CNLP. The same primal and dual initial values were used for CNLP and MINOS. All dual variables are equal to zero initially.

For the stopping parameter ϕ , values were tested in this range, where CNLP usually provides adequate precision for the solution. Thereby we report, for all models, tests

Table II
Runtime (sec) Using CNLP and MINOS: A
Comparison Using Relative Tolerances $\phi = 10^{-4}$ and
 $\phi = 10^{-6}$ of CNLP, as well as Default Tolerances
(M-Default) of MINOS

Problem	$\phi = 10^{-4}$		$\phi = 10^{-6}$		M-Default	
	CNLP	MINOS	CNLP	MINOS	CNLP	MINOS
US1	32	10	42	10	240	11
US4C	90	112	521	143	6259	160
US4E	86	131	622	150	4899	171
US8C	215	419	392	562	1126	580
US8E	220	548	411	739	864	736
US16C	597	1752	1029	3492	2728	3450
US16E	429	2260	721	4909	1796	6606
JMU	114	210	216	230	407	236
LTAREF	191	2067	587	2659	3689	2821
LTAPTO	261	1350	548	2465	1034	2754
LTASCC	275	1323	899	4124	3739	4057
LTASGE	342	1287	901	2986	3548	3005
11RT	390	3432	1452	5445	11571	5502
ATL	689	3932	2223	*	7883	*
ATLX	637	4466	3851	11054	10805	11560
ATLR	1558	2904	6275	*	14290	*
Average	382	1637	1293	2783	4679	2974

*Failure

with $\phi = 10^{-4}$ and $\phi = 10^{-6}$. To compare the results, GAMS/MINOS was modified to employ the same stopping test as CNLP. For an additional comparison, CNLP was modified to employ the default optimality test of MINOS.

For these three comparisons, Tables II–IV show the run time, number of iterations, and relative error in the optimal objective function value. All solution times refer to

Table III
Number of iterations using CNLP and MINOS: A
Comparison Using Relative Tolerances $\phi = 10^{-4}$ and
 $\phi = 10^{-6}$ of CNLP, as well as Default Tolerances
(M-Default) of MINOS

Problem	$\phi = 10^{-4}$		$\phi = 10^{-6}$		M-Default	
	CNLP	MINOS	CNLP	MINOS	CNLP	MINOS
US1	1717	958	2300	968	13179	969
US4C	1037	3000	6874	3668	85306	3338
US4E	863	3000	6654	3336	52635	3672
US8C	1193	5160	2340	6514	6699	6515
US8E	1018	5400	1950	6869	4039	6872
US16C	1605	9840	3069	15442	8256	15445
US16E	975	9540	1678	16451	4127	20344
JMU	1392	2280	2617	2422	4781	2423
LTAREF	1261	27900	3950	33989	24041	35765
LTAPTO	1638	16620	3457	27180	6715	29446
LTASCC	1787	17280	6305	44363	25192	44363
LTASGE	2163	17160	5628	35107	22820	35107
11RT	1295	19260	5255	27468	42643	27468
ATL	2467	29340	8058	*	28875	*
ATLX	1877	25860	11179	53520	34071	54793
ATLR	5096	18600	19083	*	41752	*
Average	1711	13199	5649	19806	25320	20465

*Failure.

Table IV

Relative Error in the Optimal Objective Function Value Using CNLP and MINOS: A Comparison Using Relative Tolerances $\phi = 10^{-4}$ and $\phi = 10^{-6}$ of CNLP, as well as Default Tolerances (M-Default) of MINOS

Problem	$\phi = 10^{-4}$		$\phi = 10^{-6}$		M-Default	
	CNLP	MINOS	CNLP	MINOS	CNLP	MINOS
US1	2.E-05	3.E-07	3.E-07	4.E-12	5.E-10	5.E-10
US4C	3.E-06	2.E-04	2.E-08	6.E-10	3.E-11	7.E-09
US4E	5.E-06	7.E-05	8.E-08	7.E-09	3.E-11	3.E-11
US8C	5.E-06	6.E-04	3.E-08	1.E-09	1.E-10	2.E-09
US8E	2.E-05	4.E-04	2.E-08	7.E-09	2.E-09	7.E-09
US16C	5.E-06	1.E-03	1.E-08	1.E-09	1.E-10	2.E-09
US16E	6.E-06	1.E-03	4.E-08	5.E-06	1.E-10	7.E-09
JMU	2.E-07	1.E-05	2.E-08	4.E-08	1.E-09	3.E-08
LTAREF	3.E-06	1.E-03	2.E-07	3.E-05	1.E-07	3.E-05
LTAPTO	3.E-05	2.E-03	2.E-07	3.E-06	1.E-09	3.E-06
LTASCC	4.E-06	5.E-03	5.E-07	7.E-08	2.E-08	7.E-08
LTASGE	4.E-06	6.E-03	5.E-09	1.E-06	1.E-09	1.E-06
11RT	2.E-05	4.E-04	5.E-08	9.E-07	1.E-09	9.E-07
ATL	1.E-05	5.E-03	1.E-07	*	4.E-09	*
ATLX	6.E-06	2.E-03	1.E-07	4.E-05	2.E-09	2.E-05
ATLR	6.E-05	5.E-03	5.E-07	*	2.E-09	*
Average	7.E-06	5.E-04	6.E-08	6.E-08	8.E-10	4.E-08

*Failure.

cpu seconds on a HP 9000/720 operating under HP-UX B.10.01.

Consider first the comparison with $\phi = 10^{-4}$. The general observation in these results is that the larger the problem, the more efficient CNLP is relative to MINOS. Only for the smallest problem, CNLP is slower.

Looking at the U.S.-problems, for instance, this may be explained as follows. Let s denote the number of scenarios in these problems. For MINOS, the number of iterations increases proportionally with s and the work per iteration increases even faster. Thus the run time for MINOS increases faster than s^2 . For CNLP, the number of iterations appears relatively independent of s , while the work per iteration increases proportionally with s . Consequently, the serial run time appears roughly proportional to s .

For the U.S.-problems, the compact formulations with CNLP tend to require more iterations and time than the larger equivalent formulations employing the nonanticipativity constraints. We believe, that this is due to a higher density of the columns in the former formulations.

In Table IV, the precision with $\phi = 10^{-4}$ for CNLP is roughly five digits as predicted, whereas the solution of MINOS with the same stopping test is less precise in most cases. The latter is explained by the fact that MINOS tends to keep variables at their bounds, and consequently our error measure (2.1) tends to be too small to justify its use in an optimality test for MINOS.

The general tendency in CNLP is that the convergence ratio gets worse along the iterations. To illustrate this consider the case with $\phi = 10^{-6}$, where the average run time for CNLP is increased by factor four as compared with the

Table V

Unit Effort (msec) of CNLP with $\phi = 10^{-4}$ and $\phi = 10^{-6}$

Problem	$\phi = 10^{-4}$	$\phi = 10^{-6}$
US1	32	42
US4C	24	144
US4E	18	133
US8C	31	57
US8E	22	42
US16C	44	77
US16E	22	37
JMU	34	64
LTAREF	25	77
LTAPTO	34	71
LTASCC	35	117
LTASGE	44	117
11RT	26	97
ATL	53	172
ATLX	41	250
ATLR	99	401
Average	36	119

previous case. Yet the run time with MINOS is longer in all except the three smallest U.S.-problems. Two of the larger problems appeared unsolvable using GAMS/MINOS. As expected, on the average, CNLP produces seven significant digits in the optimal objective function value.

For the runs employing the default optimality test of MINOS, precision was further improved. For several problems CNLP still shows improvement over MINOS, but for some smaller problems the opposite is quite dramatically true.

Finally, based on the serial run times for CNLP, we also calculated the amount of time required per unit of problem size where a problem's size is defined to be $m + n$. Such unit efforts are shown in Table V for CNLP with $\phi = 10^{-4}$ and $\phi = 10^{-6}$. The results illustrate that the unit computational effort is relatively independent of the problem size. This has major implications for the algorithm's behavior when implemented in parallel.

In closing, we note that backward steps, which can occur when going from the completion phase to the trial phase and can greatly increase the expense of calculating the correct perturbation mapping, usually occur only once every two or more iterations in the problems we investigated. Further, the frequency with which backward steps occur actually declines as the solution is approached, owing to the increased accuracy of the linear approximation.

4. CONCLUSIONS

A recent approach for saddle point computation (Kallio and Ruszczyński 1994b) has been specialized to solve large-scale convex optimization problems with differentiable objective and constraint functions. This iterative method proceeds in directions determined by gradients of the standard Lagrangian. Gradient evaluation takes place at perturbed points. A central topic in this article is to propose a perturbation procedure, which yields an efficient

implementation in practical applications. In order to ensure efficiency, a scaling procedure was devised as well. The resulting method is well suited to massively parallel computing.

An experimental code CNLP embedded into GAMS was tested in a serial computer on 16 nonlinear problems representing multistage stochastic optimization of the U.S. energy economy, and dynamic and regionalized macro-economic models. These preliminary tests indicate that, for large problems with adequate precision requirements, our method is faster than MINOS. Thus the method can be very efficient for large problems, even in serial computing, but especially in parallel computing environments.

APPENDIX PROOF OF THEOREM 1

Proof. It is well known (see, e.g., Rockafellar 1970) that if $(\hat{x}, \hat{y})$ is a saddle point in (6), then $\hat{x}$ and $\hat{y}$ is an optimal solution for (1)–(3). Furthermore, under our constraint qualification, by Bazaraa and Shetty's (1979) Theorem 6.2.5, such optimal solutions and saddle points are equivalent. Therefore, it suffices to show, that the algorithm converges to a saddle point, if one exists. The rest of the proof is adopted from Kallio and Ruszczyński (1994b).

Let (x^*, y^*) be a saddle point of L on $X \times Y$. We define

$$W_k = \|x^k - x^*\|^2 + \|y^k - y^*\|^2. \quad (\text{A.1})$$

We shall prove that our algorithmic mapping decreases the distance W_k whenever (x^k, y^k) is not a solution. First, note that condition 3 of Lemma 1 implies that the method can stop only at a saddle point. Therefore one can assume from now on that the sequence (x^k, y^k) is infinite.

Since the projection on X is nonexpansive,

$$\|x^{k+1} - x^*\|^2 \leq \|x^k + \tau_k d_x^k - x^*\|^2 = \|x^k - x^*\|^2 + 2\tau_k \langle d_x^k, x^k - x^* \rangle + \tau_k^2 \|d_x^k\|^2. \quad (\text{A.2})$$

Since $x^* - x^k \in X_k$

$$\langle L_x(x^k, \eta^k) - d_x^k, x^* - x^k \rangle \leq 0.$$

By convexity,

$$\langle L_x(x^k, \eta^k), x^k - x^* \rangle \geq L(x^k, \eta^k) - L(x^*, \eta^k).$$

Employing the above inequalities in (A.2) yields

$$\|x^{k+1} - x^*\|^2 \leq \|x^k - x^*\|^2 - 2\tau_k [L(x^k, \eta^k) - L(x^*, \eta^k)] + \tau_k^2 \|d_x^k\|^2.$$

Likewise, by obvious symmetry, one obtains

$$\|y^{k+1} - y^*\|^2 \leq \|y^k - y^*\|^2 + 2\tau_k [L(\xi^k, y^k) - L(\xi^k, y^*)] + \tau_k^2 \|d_y^k\|^2.$$

Adding the last two inequalities one concludes that

$$W_{k+1} \leq W_k - 2\tau_k [L(x^k, \eta^k) - L(x^*, \eta^k) - L(\xi^k, y^k) + L(\xi^k, y^*)] + \tau_k^2 \|d^k\|^2. \quad (\text{A.3})$$

The saddle point conditions imply that $L(x^*, \eta^k) \leq L(\xi^k, y^*)$. Therefore, taking into account condition 2 of Lemma 1, (A.3) can be rewritten as follows:

$$W_{k+1} \leq W_k - 2\tau_k E_k + \tau_k^2 \|d^k\|^2. \quad (\text{A.4})$$

Substituting (1.10) one gets

$$W_{k+1} \leq W_k - \frac{\gamma(2 - \gamma) E_k^2}{\|d^k\|^2}. \quad (\text{A.5})$$

Thus the sequence $\{W_k\}$ is nonincreasing and

$$\lim_{k \rightarrow \infty} \frac{E_k^2}{\|d^k\|^2} = 0. \quad (\text{A.6})$$

Since W_k is bounded, the sequence $\{(x^k, y^k)\}$ has an accumulation point $(\hat{x}, \hat{y})$. By condition 1 of Lemma 1, $\{d^k\}$ is bounded and, by (A.6), $\lim_{k \rightarrow \infty} E_k = 0$. Therefore, by condition 3 of Lemma 1, $(\hat{x}, \hat{y})$ is a saddle point of L and one can use it instead of (x^*, y^*) in (A.1). Then, from (A.5) one concludes that the distance to $(\hat{x}, \hat{y})$ is nonincreasing. Consequently, $(\hat{x}, \hat{y})$ is the only accumulation point of the sequence $\{(x^k, y^k)\}$. $\square$

ACKNOWLEDGMENTS

The authors wish to thank Andrzej P. Wierzbicki for fruitful discussions concerning the perturbation scheme, and in particular for referring us to the Goldstein test for line search. Steve Dirkse kindly helped us to set up a suitable version of GAMS/MINOS for computational tests. Constructive and encouraging comments by Yuri Ermoliev and Michael Saunders are gratefully acknowledged as well. The authors are grateful to an anonymous associate editor and two referees for valuable comments, which we believe improved the revised manuscript considerably. Thanks to Alan Manne for naming our code!

REFERENCES

- Anstreicher, K. M., J. P. Vial. 1994. On the convergence of an infeasible primal-dual interior-point method for convex programming. *Optimiz. Methods and Software* 3 273–283.
- Bazaraa, M. S., C. M. Shetty. 1979. *Nonlinear Programming*. John Wiley & Sons, New York.
- Brooke, A., D. Kendrick, A. Meeraus. 1992. *GAMS: User's Guide, Release 2.25*. Scientific Press, New York.
- Dirkse, S. P., M. C. Ferris, P. V. Preckel, T. Rutherford. 1994. The GAMS callable program library for variational and complementary solvers. Math. Programming Technical Report 94-07. Computer Sciences Dept., University of Wisconsin.
- Fletcher, R. 1987. *Practical Methods of Optimization*. John Wiley & Sons, New York.
- Kallio, M., A. Ruszczyński. 1994a. Parallel solution of linear programs via Nash equilibria. Working Paper WP-94-15, IIASA, Laxenburg, Austria.
- , —. 1994b. Perturbation methods for saddle point computation. Working Paper WP-94-38, IIASA, Laxenburg, Austria.

- , Salo. 1994. Tatonnement procedures for linearly constrained convex optimization. *Mgmt. Sci.* **40**, 788–797.
- Manne, A. S., R. G. Richels. 1992. *Buying Greenhouse Insurance: The Economic Costs of CO₂ Emission Limits*. MIT Press, Cambridge, MA.
- , ——. 1995. The greenhouse debate—economic efficiency, burden sharing and hedging strategies. Stanford University and Electric Power Research Institute, Stanford, CA.
- Murtagh, B. A., M. A. Saunders. 1987. MINOS 5.1 user's guide. Technical Report SOL 83-20R, Stanford University, Stanford, CA.
- Nesterov, Y. E. 1994. Infeasible start interior point primal-dual methods in nonlinear programming. Technical Report. CORE, Belgium.
- , A. S. Nemirovsky. 1994. *Interior Point Polynomial Algorithms in Convex Programming: Theory and Algorithms*. SIAM Publications, Philadelphia, PA.
- Olsen, T. 1994. Greenhouse gas abatement—joint maximization under uncertainty. Doctoral Dissertation. Dept. of Operations Research, Stanford University, Stanford, CA.
- Rockafellar, R. T. 1970. *Convex Analysis*. Princeton University Press, Princeton, NJ.
- Rosa, C. H. 1994. Pathways of economic development in an uncertain environment. Working Paper WP-94-41, IIASA, Laxenburg, Austria.
- Rzhevskii, S. V. 1994. The monotone ϵ -subgradient method for finding the saddle points of convex-concave functions. *Cyber. Systems Analy.* **29** 546–555.
- . 1990. The conditional ϵ -subgradient method for the simultaneous solution of the original and the dual convex programming problem. *Cybernetics* **26** 265–275.
- . 1991. On the structure of the conditional ϵ -subgradient method for the simultaneous solution of the direct and the dual programming problem. *Soviet Math. Doklady* **41** 347–351.
- Schrattenholzer, L., A. Schäfer. 1995. World regional scenarios described with the 11R model of energy-economy-environment interactions. IIASA Working Paper. International Institute for Applied Systems Analysis, Laxenburg, Austria.
- Sen, S., H. D. Sherali. 1986. A class of convergent primal-dual subgradient algorithms for nondifferentiable optimization. *Math. Programming* **35** 279–297.
- , D. Yakowitz. 1986. A primal-dual subgradient method for time staged capacity expansion. *European J. Oper. Res.* **27** 301–312.
- Zabotin, I. Y. 1988. A subgradient method for finding a saddle point of a convex-concave function. Kazanskii Universitet. *Issledovaniya po Prikladnoi Matematike* **15** 6–12.